

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA0101
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I Mangali Veera Naga Sai (192425253) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Veera Naga Sai

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.

9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

Banking Loan Eligibility Expert System Using Prolog

1. Title, Abstract & Introduction

Project Title: Prolog-Based Banking Loan Eligibility Decision-Support Expert System

Abstract

The proposed system is a Prolog-based expert system designed to pre-screen customers for loan eligibility. The system evaluates important customer information such as income, credit score, employment status, existing loans, and repayment history. Using Prolog facts, rules, unification, backward chaining, and backtracking, the system determines whether a customer is eligible, conditionally eligible, or not eligible for a loan. It also provides reasons supporting the decision.

Introduction

Banks receive a large number of loan applications. Manually checking every application can be time-consuming and may result in inconsistent initial screening. An expert system can automate the pre-screening stage by applying predefined banking rules.

The proposed system does not replace a bank's complete credit assessment. Instead, it provides an initial decision based on the information supplied.

2. Problem Statement & Objectives

Problem Statement

A bank needs an automated system to evaluate customers before processing their loan applications. The system should analyze:

- Monthly income
- Credit score
- Employment status
- Existing loans
- Repayment history

Based on these factors, it should determine loan eligibility and explain the reason.

Inputs

Income

Credit Score

Employment Status

Existing Loans

Repayment History

Outputs

Eligible

Conditionally Eligible

Not Eligible

Reason for decision

Stakeholders

- Customers
- Bank loan officers
- Credit department
- Bank management

Objectives

1. Develop a Prolog-based expert system.
2. Represent banking knowledge using facts and rules.
3. Determine loan eligibility automatically.
4. Provide reasons for each decision.
5. Demonstrate backward chaining, unification, and backtracking.
6. Reduce manual effort during initial loan screening.

3. Domain Analysis & Knowledge Acquisition

Domain

The selected domain is **banking and loan pre-screening**.

Important Entities

Entity	Description
Customer	Person applying for loan
Income	Customer's monthly income
Credit Score	Measure of creditworthiness
Employment	Employment condition
Existing Loans	Current financial obligations

Entity	Description
Repayment History	Previous loan repayment behavior
Loan	Financial product requested
Decision	Eligibility result

Knowledge Acquisition

The rules can be developed from:

- General banking loan-screening practices
- Credit-risk concepts
- Discussions with banking/finance professionals
- Publicly available financial-literacy and lending guidance
- Academic material on expert systems

For this academic project, the thresholds are illustrative rules, not actual bank lending policies.

4. Knowledge Representation & Production Rules

Prolog Facts

Customer information can be represented using predicates.

income(1001, 60000).

credit_score(1001, 750).

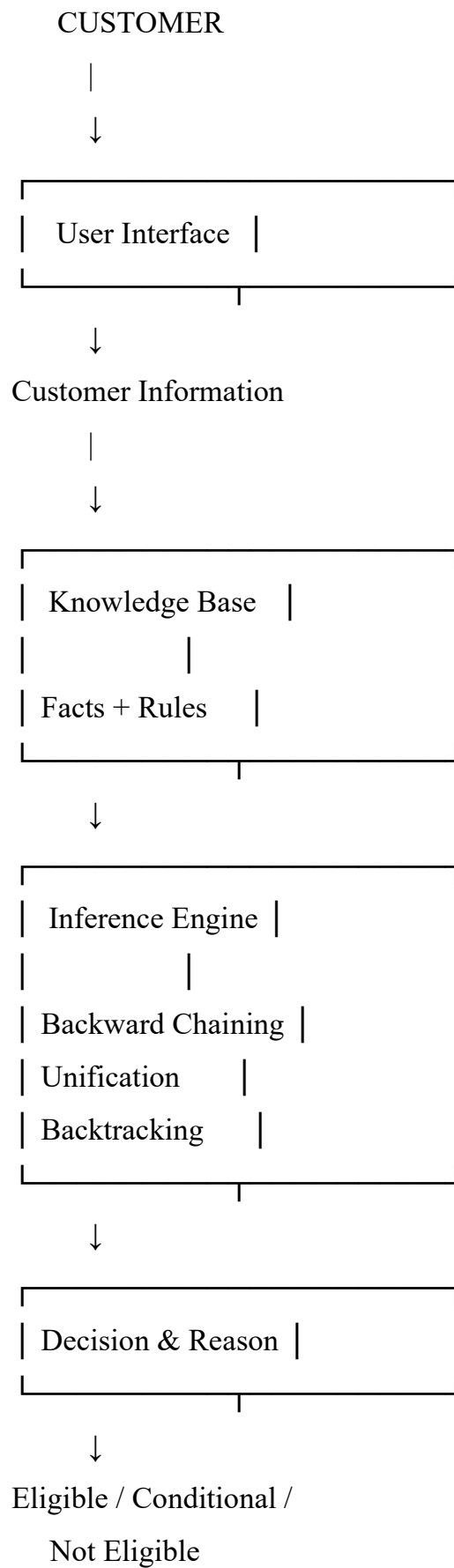
employment(1001, stable).

existing_loans(1001, low).

repayment_history(1001, good).

Here, 1001 represents the customer ID.

5. Expert System Architecture



Components

1. User Interface

Collects customer information.

2. Knowledge Base

Contains Prolog facts and rules.

3. Inference Engine

Uses Prolog's logical reasoning mechanisms.

4. Explanation Module

Displays why the customer was accepted or rejected.

6. Inference Mechanism & Algorithms

A. Forward Chaining

Forward chaining starts with **known facts** and derives conclusions.

Example:

Income = 60000

Credit score = 750

Employment = stable

Existing loans = low

Repayment = good

The system derives:

Sufficient income

↓

Good credit

↓

Stable employment

↓

Low debt

↓

Good repayment

↓

Eligible

7. Prolog Implementation

The system can be implemented using **SWI-Prolog**.

Complete Simple Prolog Program

% Customer facts

income(1001, 60000).

credit_score(1001, 750).

employment(1001, stable).

existing_loans(1001, low).

repayment_history(1001, good).

income(1002, 25000).

credit_score(1002, 720).

employment(1002, stable).

existing_loans(1002, low).

repayment_history(1002, good).

income(1003, 50000).

credit_score(1003, 550).

employment(1003, stable).

existing_loans(1003, low).

repayment_history(1003, good).

% Rules

good_credit(C) :-

 credit_score(C, S),

 S >= 700.

sufficient_income(C) :-

 income(C, I),

$I \geq 30000$.

stable_employment(C) :-
 employment(C, stable).

low_debt(C) :-
 existing_loans(C, low).

good_repayment(C) :-
 repayment_history(C, good).

eligible(C) :-
 sufficient_income(C),
 good_credit(C),
 stable_employment(C),
 low_debt(C),
 good_repayment(C).

not_eligible(C) :-
 credit_score(C, S),
 $S < 600$.

not_eligible(C) :-
 repayment_history(C, poor).

reason(C, 'Low credit score') :-
 credit_score(C, S),
 $S < 600$.

reason(C, 'Poor repayment history') :-
 repayment_history(C, poor).

reason(C, 'High existing debt') :-
existing_loans(C, high).

reason(C, 'Insufficient income') :-
income(C, I),
 $I < 30000$.

decision(C, eligible) :-
eligible(C).

decision(C, not_eligible) :-
not_eligible(C).

Example Queries

?- eligible(1001).

Output:

true.

Query:

?- decision(1003, D).

Output:

D = not_eligible.

Query:

?- reason(1003, R).

Output:

R = 'Low credit score'.

8. Test Cases & Results

Minimum **5 industry-based test cases** can be demonstrated.

Test Case	Income	Credit Score	Employment	Existing Loans	Repayment	Expected
TC1	₹60,000	750	Stable	Low	Good	Eligible
TC2	₹25,000	720	Stable	Low	Good	Not Eligible
TC3	₹50,000	550	Stable	Low	Good	Not Eligible
TC4	₹55,000	730	Stable	High	Good	Not Eligible/Review
TC5	₹50,000	720	Stable	Low	Poor	Not Eligible

TC1 – Successful Applicant

?- eligible(1001).

Output:

true.

Reason:

Income is sufficient

Credit score is good

Employment is stable

Existing loans are low

Repayment history is good

TC2 – Low Income

?- sufficient_income(1002).

Output:

false.

Reason:

Income is below the required threshold.

TC3 – Low Credit Score

?- not_eligible(1003).

Output:

true.

Reason:

Low credit score.

TC4 – High Existing Debt

If the customer has:

Income = 55000

Credit Score = 730

Employment = stable

Existing Loans = high

Repayment = good

The system should flag the application for **additional review** rather than automatically approve it.

TC5 – Poor Repayment History

If:

Repayment History = poor

then:

?- not_eligible(Customer).

can succeed through the poor-repayment rule.

9. Analysis, Limitations & Future Enhancements

Analysis

The system provides:

- Fast initial screening
- Consistent application of rules
- Explainable decisions
- Easy modification of rules
- Logical reasoning through Prolog

Limitations

1. It uses simplified rules.
2. It does not calculate actual loan interest.
3. It does not access real-time credit-bureau data.
4. It does not consider all banking regulations.

5. It cannot replace a professional credit assessment.
6. Fixed thresholds may not suit every type of loan.

Future Enhancements

The system can be improved by adding:

- Real credit-bureau integration
- Debt-to-income ratio
- Loan amount requested
- Loan tenure
- Interest-rate calculation
- Fraud detection
- More detailed risk categories
- Web/mobile interface
- Machine-learning-based risk prediction
- Database integration

10. Conclusion & References

Conclusion

The **Banking Loan Eligibility Expert System** successfully demonstrates how Prolog can be used to develop a rule-based decision-support system. Customer information such as income, credit score, employment status, existing loans, and repayment history is represented using **Prolog facts and predicates**.

The inference engine applies logical rules to determine loan eligibility. **Backward chaining** is demonstrated through Prolog queries, while **unification and backtracking** support the reasoning process. The system also provides explanations for decisions, making it more transparent and suitable for an academic demonstration of an expert system.

The project demonstrates the practical application of **Artificial Intelligence, knowledge representation, and logic programming in the banking domain**.

References

1. Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Pearson.
2. SWI-Prolog Documentation, *SWI-Prolog Reference Manual*.

3. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
4. Elaine Rich, Kevin Knight and Shivashankar B. Nair, *Artificial Intelligence*, McGraw-Hill.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference traces.	7–8% – Provides relevant test cases with mostly accurate results.	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				